<table>
<tr><td rowspan="2"></td><td colspan="3">Politechnika Śląska</td><td rowspan="2">
Politechnika
Śląska</td></tr>
<tr><td colspan="3">Katedra Grafiki, Wizji komputerowej
i Systemów Cyfrowych</td></tr>
<tr><td>Academic year</td><td>Type of studies:</td><td>Course:</td><td>Group</td><td>Section</td></tr>
<tr><td>2025/2026</td><td>SSI</td><td>BIAI</td><td>2</td><td>3</td></tr>
<tr><td>Supervisor:</td><td colspan="2">mgr inż. Anna Daniłowicz</td><td colspan="2">Classes: (day, hour)</td></tr>
<tr><td rowspan="2">Section members:</td><td colspan="2" rowspan="2">Błażej Jamrozik
Bartosz Kępa</td><td colspan="2">11.06.26</td></tr>
<tr><td rowspan="2">emails:</td><td colspan="2" rowspan="2">bj311378@student.polsl.pl
bk311386@student.polsl.pl</td><td colspan="2" rowspan="2">10:00 – 11:30</td></tr>
<tr><td colspan="5">Report</td></tr>
<tr><td colspan="5">Subject: Detection of AI-Generated Images in Social Media</td></tr>
</table>

1. Introduction

The rapid advancement of generative AI models has dramatically lowered the barrier to creating photorealistic synthetic images. These models are now capable of producing content indistinguishable from authentic photographs, which has significant implications for the integrity of information shared on social media platforms.

This project addresses the problem through binary image classification (determining whether a given image is REAL or AI GENERATED) using deep convolutional neural networks (CNNs). The work compares a custom baseline CNN trained from scratch with a fine-tuned ResNet50 model leveraging transfer learning, evaluating both architectures across multiple datasets to assess generalisation capability.

2. Analysis of the task

2.1 Possible Approaches and Selected Methodology

Several approaches to AI-generated image detection were considered:

- **Metadata and provenance analysis**: Inspects EXIF data and C2PA content credentials embedded in images. High precision when metadata is intact, but easily circumvented by stripping or forging metadata.
- **Pixel-level statistical methods**: Analyses noise patterns and pixel co-occurrence statistics. Computationally lightweight but limited in generalisation across different generator architectures.
- **Deep learning classification** (selected): End-to-end CNN-based binary classifiers that learn discriminative visual features directly from data. Robust to diverse generator styles and scalable to large datasets.

The deep learning approach was selected due to its demonstrated state-of-the-art performance on visual classification tasks and its ability to generalise across heterogeneous image sources. Two architectures were implemented and compared:

- **Baseline CNN**: A custom three-block convolutional network trained from scratch, serving as a reference model.
- **ResNet50** with transfer learning: A pretrained ResNet50 fine-tuned on the binary detection task.

2.2 Datasets

The project utilised six datasets sourced from Kaggle and Hugging Face, providing diverse coverage of generator styles and image domains. This multi-source approach enables cross-dataset evaluation and tests model robustness against distribution shift.

Dataset	Source	Role
PARVESHI	Parveshiiii/AI-vs-Real	Primary training dataset
LuckyCow	LuckyCow/Ai_vs_real_web_images	Cross-dataset evaluation
NanoBanana	julienlucas/midjourney-dalle-sd-nanobananapro-dataset	Cross-dataset evaluation
Kaggle 1	tristanzhang32/ai-generated-images-vs-real-images	Cross-dataset evaluation
Kaggle 2	Hemg/AI-Generated-vs-Real-Images-Datasets	Cross-dataset evaluation
Kaggle 3	rhythmghai/ai-vs-real-images-dataset	Cross-dataset evaluation

The **Parveshiiii/AI-vs-Real** dataset served as the default and primary training dataset for the model. The dataset, created by Parvesh Rawal and hosted on Hugging Face, is a publicly available image classification dataset designed for research on detecting AI-generated images and distinguishing them from authentic photographs.

The dataset contains **13,999 images** divided into two balanced classes:

- **AI-generated images** (label 0),
- **Real photographs** (label 1).

The complete dataset occupies approximately **2.16 GB** of storage space.

A key characteristic of the dataset is its balanced class distribution, ensuring a similar number of AI-generated and real images. This reduces the risk of classification bias and allows deep learning models to learn both classes equally during training.

2.3 Tools and Frameworks

Several frameworks were considered for implementing the deep learning pipeline. The primary candidates were TensorFlow/Keras, PyTorch, and JAX.

TensorFlow/Keras is a mature, production-oriented framework with strong deployment tooling. Keras provides a high-level API that reduces boilerplate. However, the ecosystem around computer vision research has shifted noticeably toward PyTorch in recent years.

JAX offers excellent performance through JIT compilation and functional transformations, making it attractive for research-level experimentation. That said, it has a steeper learning curve and a less mature ecosystem of pretrained vision models, which would complicate the transfer learning component of this project.

PyTorch with torchvision was selected as the framework of choice due to its practical advantages. Torchvision provides ready-to-use pretrained ResNet-50 weights together with standard dataset

handling and image transformation utilities. Moreover, its explicit training loop design facilitates fine-grained control and easy instrumentation during experiments.

The entire project was implemented in Python using the following stack:

Component	Library / Tool	Purpose
Deep learning framework	PyTorch & torchvision	Model definition, training, inference
Data manipulation	NumPy & Pandas	Numerical computation, result aggregation
Evaluation & visualisation	scikit-learn & Matplotlib	Metrics, confusion matrices, plots
Web application	Streamlit	Interactive deployment interface
Version control	GitHub	Collaborative development and code hosting

3. Software Specification

3.1 Internal Specification

The project is structured as a collection of Python scripts. The key components are as follows.

3.1.1 Architecture: Baseline CNN

The Baseline CNN was implemented as a sequential convolutional network with three progressively deeper feature extraction blocks, followed by a fully connected classification head:

Layer	Configuration	Description
Block 1	Conv2d(3→32) + ReLU + MaxPool2d(2)	Edge and texture feature extraction
Block 2	Conv2d(32→64) + ReLU + MaxPool2d(2)	Mid-level shape and pattern features
Block 3	Conv2d(64→128) + ReLU + MaxPool2d(2)	High-level semantic features
Flatten	128 × 28 × 28 = 100,352 features	Spatial-to-vector conversion
FC + Dropout	Linear(100352 → 256) + ReLU + Dropout(0.5)	Dense representation with regularisation

Output	Linear(256 → 2)	Binary classification logits: REAL / AI GENERATED

3.1.2 Architecture: ResNet50 with Transfer Learning

The production model reuses a ResNet50 pretrained on ImageNet (1.2M images, 1000 classes). The backbone was partially frozen to preserve low-level feature representations, while the final residual block and classification head were fine-tuned on the detection task:

- **Pretrained weights**: ImageNet - rich visual priors reduce the number of training epochs required.
- **Frozen layers**: All backbone parameters are frozen by default (requires_grad=False)
- **Unfrozen block**: layer4 (final residual block) is fine-tuned for the target domain.
- **Replaced FC layer**: Original 1000-class head replaced with Linear(2048 → 2) for binary classification.

3.1.3 Training scripts

A standard training loop was implemented in PyTorch, following a typical deep learning workflow. Input data was loaded using torch.utils.data.DataLoader, which handles batching and shuffling. Image preprocessing was defined as a sequence of transformations using transforms.Compose. During training, this included resizing, random horizontal flips, random rotations, color jitter, conversion to tensors, and normalization using ImageNet statistics. During evaluation, only resizing, tensor conversion, and normalization were applied. The loss function used during training was cross-entropy loss (nn.CrossEntropyLoss), while optimization was performed using the Adam optimizer (torch.optim.Adam) with a configurable learning rate. During each epoch, training loss, training accuracy, validation loss, and validation accuracy were recorded in Python lists for later visualization and analysis of learning curves.

3.1.4 Evaluation scripts

The evaluation phase was conducted in inference mode using model.eval() and torch.no_grad(), which ensured that no gradients were computed and that the model parameters remained unchanged. The network outputs in the form of logits were converted into class predictions using argmax, and these predictions were then compared with the ground-truth labels stored in flat lists. The final assessment of model performance was carried out using scikit-learn metrics, specifically accuracy_score, f1_score, and classification_report. Evaluation was performed on the test DataLoader returned by get_loaders().

3.1.5 Data structures

During training, model history is stored in a Python dictionary containing the keys epoch, loss, accuracy, val_loss, and val_accuracy, each mapped to a list of per-epoch floating-point values. This history is used by Matplotlib in the reporting module to generate training curves, which are then displayed in the Streamlit interface as a saved image. Trained model parameters are saved in .pth files, as ordered dictionaries mapping layer names to their associated weight and bias tensors.

3.2 External Specification

The application is deployed as a Streamlit web app providing:

- **Model selection panel**: choose between Baseline CNN and ResNet50 (fine-tuned).
- **Optional heatmap mode**: Grad-CAM visualisation highlighting regions that influenced the prediction.
- **Image upload widget**: multiple images can be queued for batch inspection.
- **Analysis result panel**: displays predicted class label (REAL / AI GENERATED), confidence percentage, and per-class probability bars.
- **Training curve display**: Loss and accuracy curves for both models accessible in the settings sidebar.

The pipeline is fully automatic. The user uploads an image and receives a prediction with confidence score without any manual intervention. This design makes the tool accessible to non-technical users.

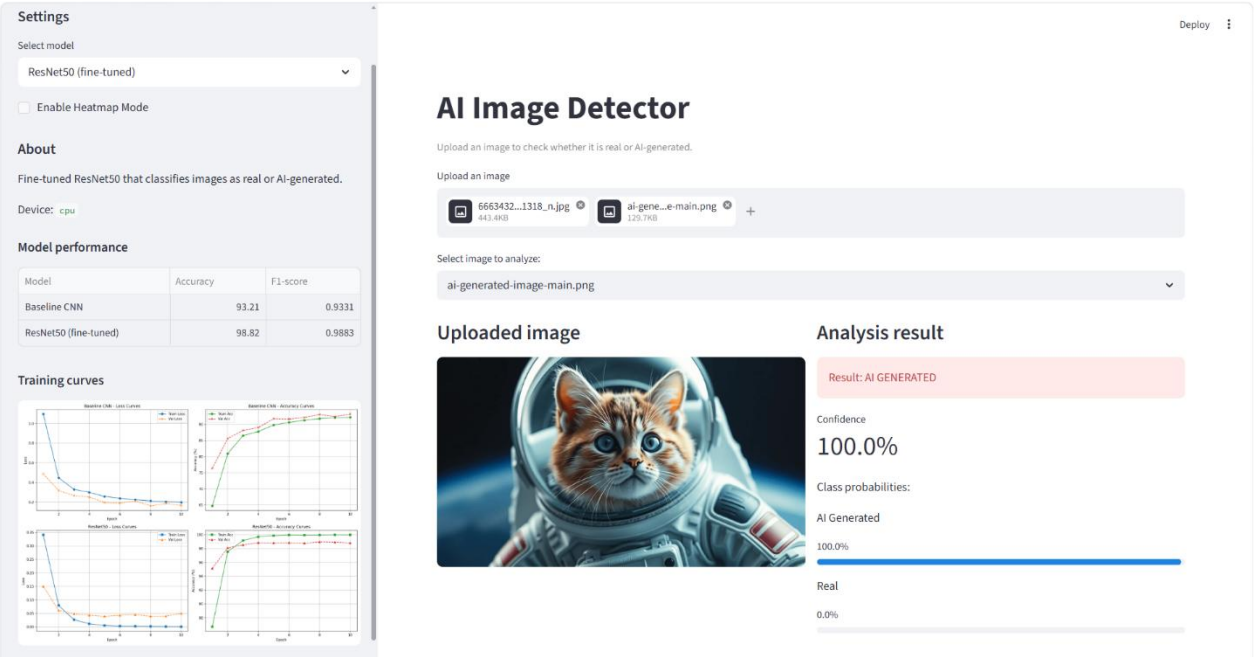

Fig. 1. The application's user interface

4. Experiments

4.1 Experimental Setup

Three parameters were varied systematically: learning rate, input image size, and number of training epochs. One parameter was changed at a time while the rest were held at the reference configuration (image size 224×224, 10 epochs, LR 0.001 for Baseline CNN and 0.0001 for ResNet50). All parameter experiments used PARVESHI as the training and validation source with a fixed split.

Learning rate was tested across {0.0001, 0.001, 0.01, 0.1} to characterise optimisation stability for both models. The expectation was that the Baseline CNN would be sensitive to high learning rates, while ResNet50 would tolerate a wider range.

Input size was tested at {224×224, 256×256, 299×299}. The motivation is that AI-generated images often contain subtle high-frequency artefacts that aggressive downsampling destroys. Larger inputs were expected to improve detection quality.

Epoch count was varied across {5, 10, 15} to assess convergence speed. ResNet50 was expected to converge early given its pretrained initialization. The Baseline CNN was expected to benefit from longer training.

Results are reported as **Accuracy** (%) and **F1-score**. Accuracy measures overall correctness on the balanced validation set; F1-score is additionally reported as it captures precision-recall balance and exposes class-biased predictions that raw accuracy can mask.

For **cross-dataset generalisation**, two configurations were compared: model trained on PARVESHI only, and model trained on all datasets combined. Both were tested against each dataset independently. This allows us to directly assess how training data diversity influences the model's robustness to out-of-distribution inputs.

4.2 Effect of Learning Rate

Four learning rates were evaluated with all other parameters at reference values:

Learning Rate	Baseline CNN Accuracy	Baseline CNN F1	ResNet50 Accuracy	ResNet50 F1
0.0001	92.89%	0.9301	98.96%	0.9897
0.001	94.39%	0.9428	99.00%	0.9899
0.01	76.46%	0.6627	99.25%	0.9925
0.1	76.46%	0.6627	99.14%	0.9914

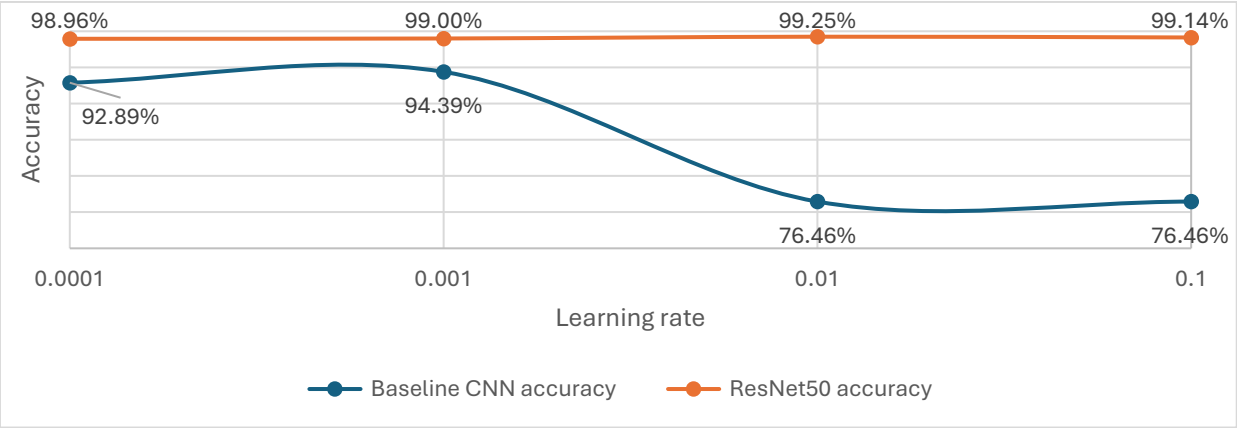

Fig. 2. Accuracy as a function of learning rate for the Baseline CNN and ResNet50

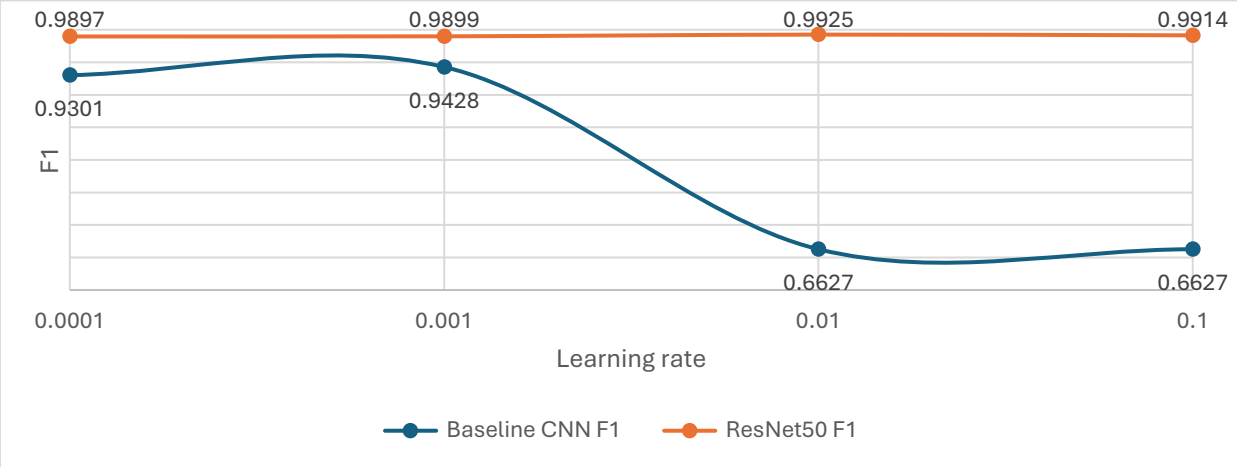

Fig. 3. F1-score as a function of learning rate for the Baseline CNN and ResNet50

The Baseline CNN degrades sharply at learning rates of 0.01 and above, with accuracy dropping to 76.5% and F1-score collapsing to 0.66. ResNet50, by contrast, maintains consistently high performance (~99%) across the entire tested range. This robustness is a direct consequence of transfer learning: pretrained weights provide a stable initialisation that tolerates a wider range of update magnitudes. The optimal learning rate for the Baseline CNN is 0.001, whereas for ResNet50 all tested learning rates yield near-optimal performance.

4.3 Effect of Input Image Size

Three sizes were tested with reference parameters:

Image Size	Baseline CNN Accuracy	Baseline CNN F1	ResNet50 Accuracy	ResNet50 F1
224×224	93.21%	0.9331	98.82%	0.9883
256×256	94.71%	0.9474	99.46%	0.9946

299×299	95.46%	0.9537	99.43%	0.9943

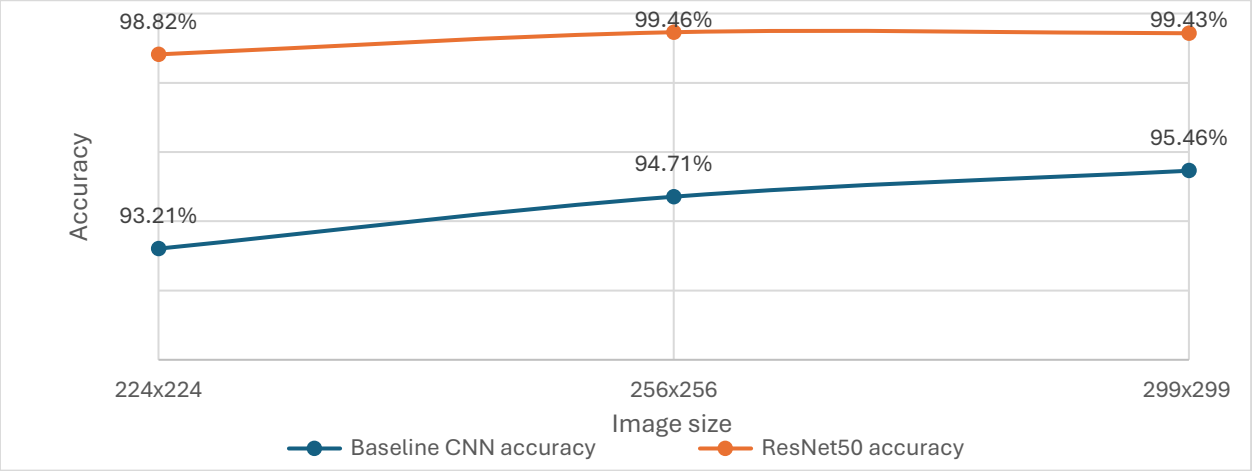

Fig. 4. Accuracy as a function of image size for the Baseline CNN and ResNet50

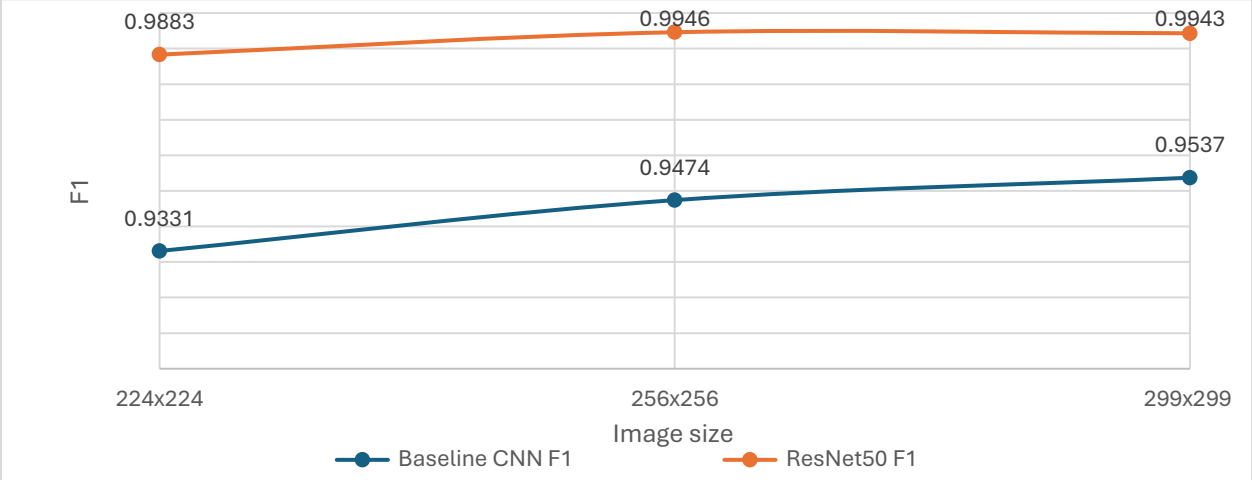

Fig. 5. F1-score as a function of image size for the Baseline CNN and ResNet50

Both models improve monotonically with increasing size. Higher pixel counts preserve finer artefacts (subtle texture inconsistencies, unnatural blurring at object boundaries, and frequency-domain signatures) that are diagnostic of generative models but lost under aggressive downsampling. The gains are more pronounced for the Baseline CNN (+2.25 pp) than for ResNet50 (+0.61 pp), since the pretrained model already extracts robust low-level features. Best overall results are achieved at 256×256 and 299×299.

4.4 Effect of Training Epochs

Training duration was varied across 5, 10, and 15 epochs:

Epochs	Baseline CNN Accuracy	Baseline CNN F1	ResNet50 Accuracy	ResNet50 F1
5	93.50%	0.9345	98.61%	0.9861
10	93.21%	0.9331	98.82%	0.9883
15	95.57%	0.9556	98.82%	0.9883

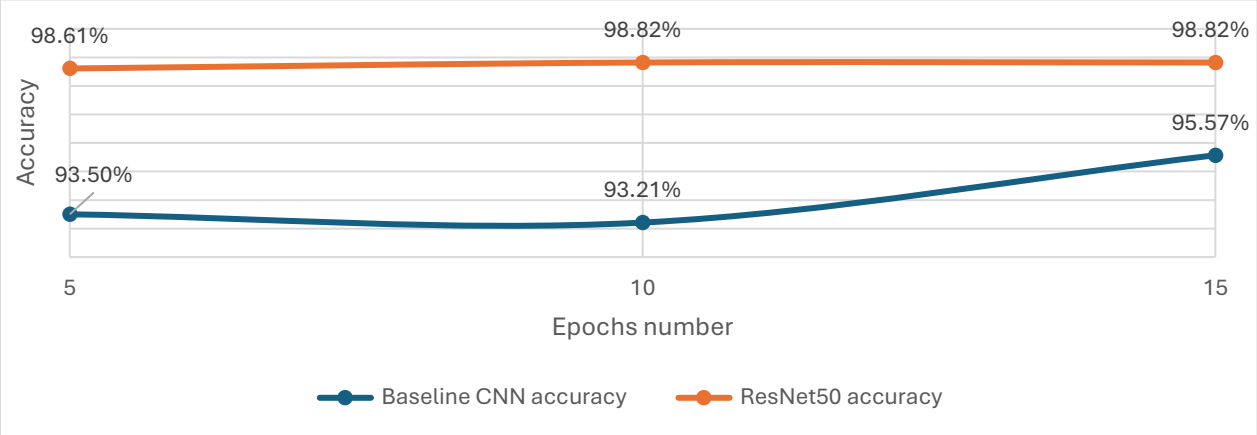

Fig. 6. Accuracy as a function of training epochs for the Baseline CNN and ResNet50

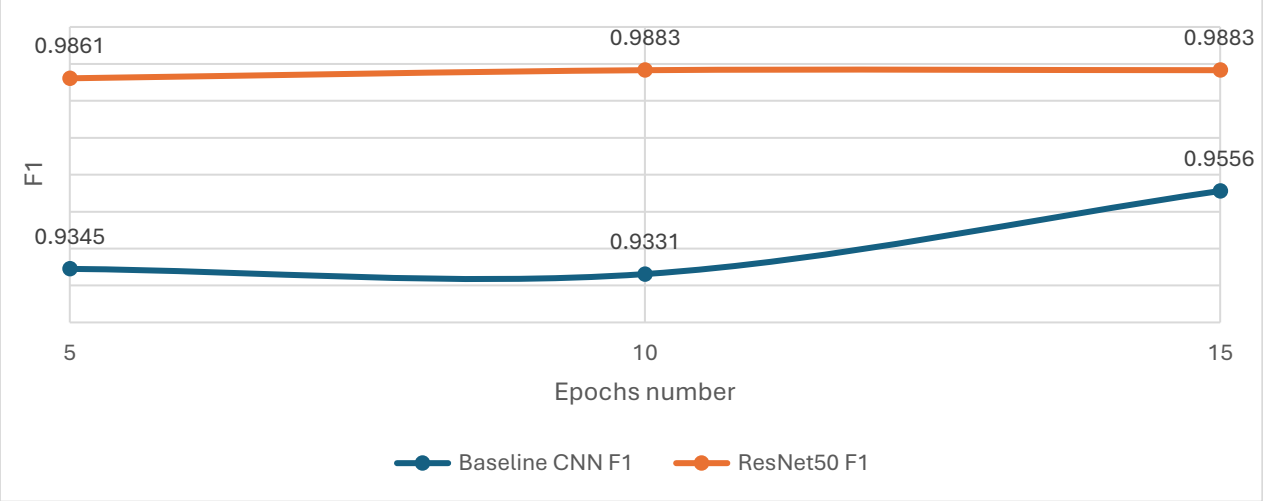

Fig. 7. F1-score as a function of the number of training epochs for the Baseline CNN and ResNet50

ResNet50 converges rapidly. Strong performance is achieved within 5 epochs thanks to the informative pretrained initialisation and plateaus by epoch 10 with no further improvement at 15. The Baseline CNN improves continuously, gaining +2.07 pp accuracy between epochs 5 and 15, but never matches ResNet50 even at 15 epochs. This demonstrates that transfer learning compresses the effective training cost without sacrificing quality. The practical optimum is 10-15 epochs.

4.5 Cross-Dataset Generalisation

This experiment tested how well a model trained exclusively on PARVESHI generalises to external datasets it has never seen. The goal was to establish a single-source baseline and identify which target domains diverge most from the training distribution.

Test Dataset	Accuracy	F1-Score
Parveshi (in-distribution)	98.67%	0.9865
NanoBanana	67.37%	0.6701
Combined	70.27%	0.6924
Kaggle 2	67.23%	0.6656
Kaggle 3	56.68%	0.5886
LuckyCow	56.23%	0.5391
Kaggle 1	47.27%	0.3155

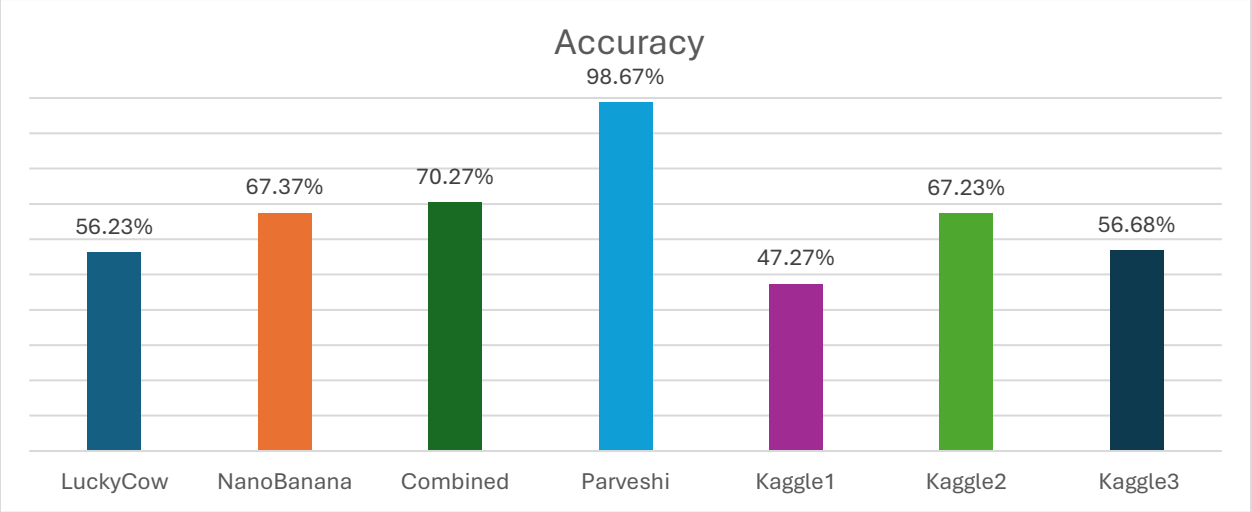

Fig. 8. Accuracy across all test datasets for the model trained on the Parveshi dataset

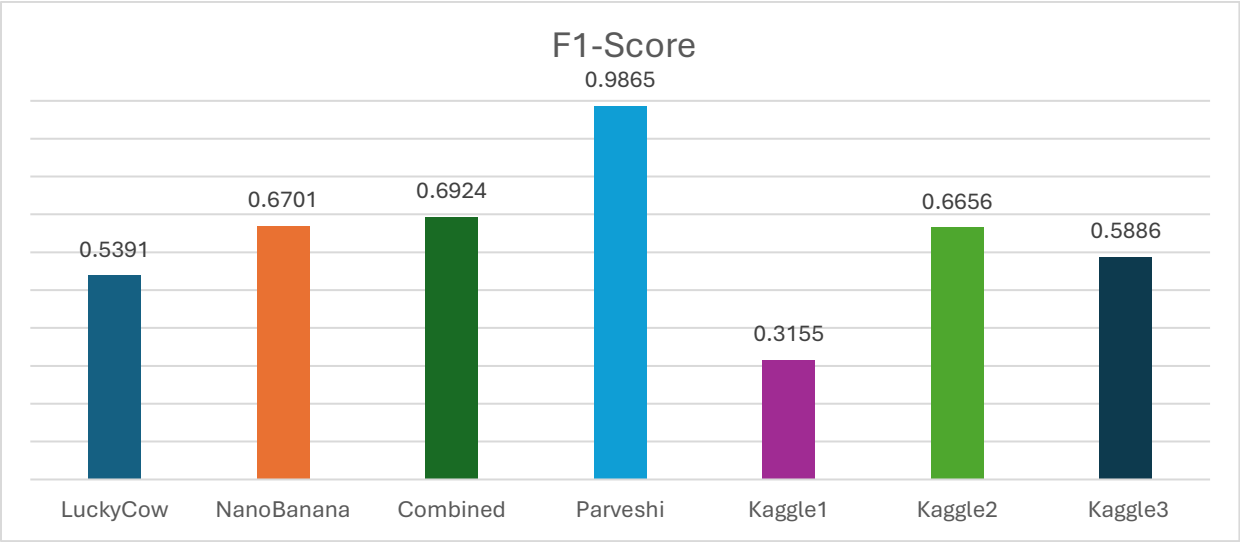

Fig. 9. F1-score across all test datasets for the model trained on the Parveshi dataset

In-distribution performance is near-perfect at 98.67%, but generalisation collapses on several external datasets. Kaggle 1 drops to 47.27% accuracy with an F1 of 0.32 (effectively below random chance for a balanced binary task). LuckyCow and Kaggle 3 sit around 56%, again near chance level. The consistently lower F1 relative to accuracy confirms the model is defaulting toward majority-class predictions rather than genuinely classifying. It has learned PARVESHI specific artefacts that simply do not transfer to other generator styles.

4.6 Cross-Dataset Generalisation: Trained on Combined Multi-Dataset

This experiment is the only case in the project where the model was not trained on PARVESHI. Instead, it was trained exclusively on a combined pool drawn from all datasets (LuckyCow, NanoBanana, Parveshi, Kaggle 1, Kaggle 2, and Kaggle 3) and then tested across all datasets. This setup was deliberately constructed to simulate a scenario where the training data is maximally diverse, allowing a clean measurement of how much training data variety alone improves generalization.

Test Dataset	Accuracy	F1-Score
Parveshi	96.47%	0.9653
NanoBanana	90.04%	0.9004
Combined	90.73%	0.9073
Kaggle 1	87.80%	0.8783
Kaggle 2	88.30%	0.8830

Kaggle 3	70.65%	0.7248
LuckyCow	66.17%	0.6565

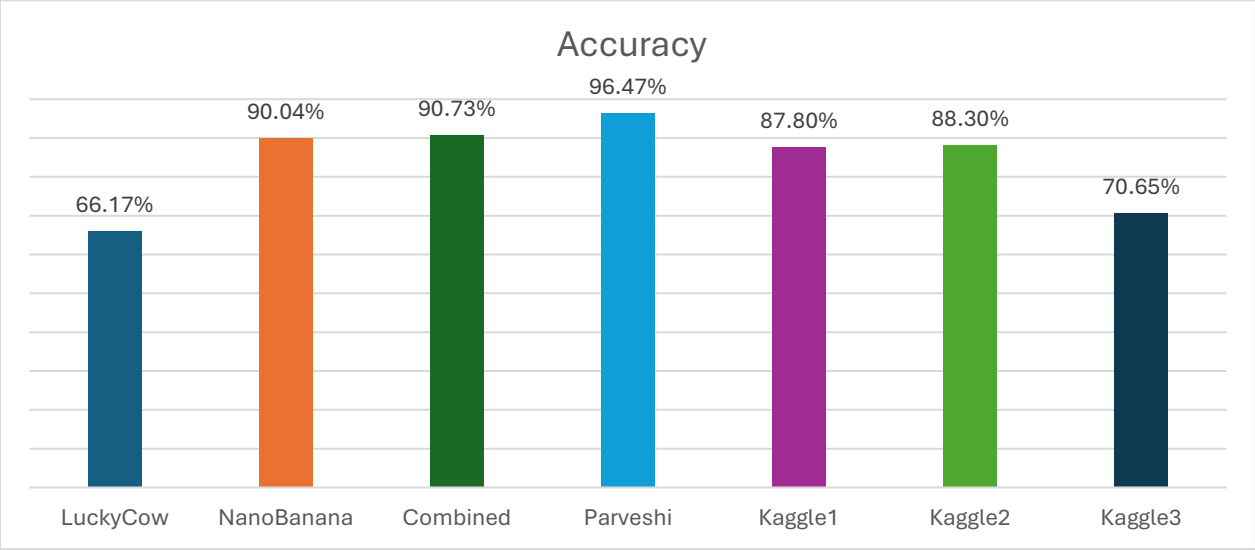

Fig. 10. Accuracy across all test datasets for the model trained on combined multi-dataset

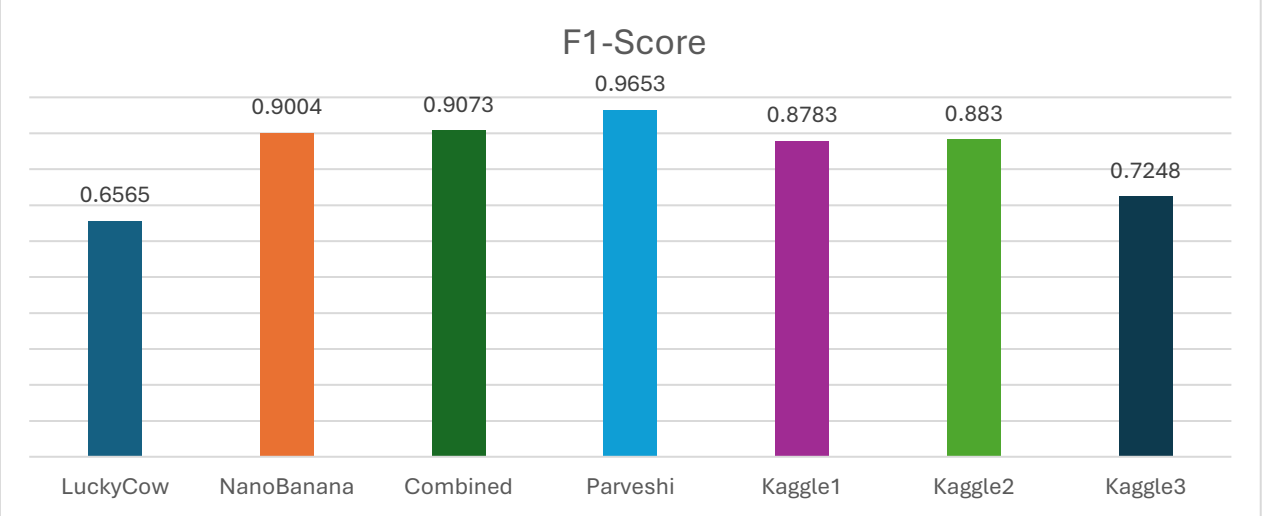

Fig. 11. F1-score across all test datasets for the model trained on combined multi-dataset

Multi-dataset training resolves most of the generalisation failures observed in the previous experiment. Kaggle 1 jumps from 47% to 88%, NanoBanana from 67% to 90%, Kaggle 2 from 67% to 88%. In-distribution performance on PARVESHI drops only marginally (-2.2 pp), which is an acceptable trade-off. LuckyCow remains the hardest target at 66%, suggesting its visual characteristics are distinct enough from all training sources to constitute a genuinely different domain. The conclusion is unambiguous: a single-source model is not deployable in practice, and multi-dataset training is a prerequisite for any robust AI image detector.

4.7 Best Model Results

This section presents the results of the best-performing configuration identified across all parameter experiments: input size 299×299, 10 training epochs, learning rate 0.001 for Baseline CNN and 0.0001 for ResNet50, trained and evaluated on the PARVESHI dataset. This configuration achieved the highest accuracy and F1-score among all tested setups for both models. Results are analysed separately for each architecture, covering training dynamics per epoch and the confusion matrix on the held-out test set. It should be noted that while the results below represent the peak in-distribution performance, generalisation to other datasets remains a known limitation of single-source training, as discussed in the cross-dataset evaluation sections.

4.7.1 Baseline CNN

The table below reports training and validation loss and accuracy across all 10 epochs:

Epoch	Train Loss	Train Accuracy	Val Loss	Val Accuracy
1	0.3979	84.44%	0.2700	88.79%
2	0.2386	90.50%	0.1914	92.57%
3	0.1989	92.34%	0.1614	93.39%
4	0.1796	93.14%	0.1453	94.61%
5	0.1585	94.18%	0.1380	94.82%
6	0.1412	94.72%	0.1283	95.07%
7	0.1311	95.18%	0.1374	94.79%
8	0.1252	95.24%	0.1197	95.68%
9	0.1245	95.41%	0.1123	96.18%
10	0.1084	95.69%	0.1193	95.46%

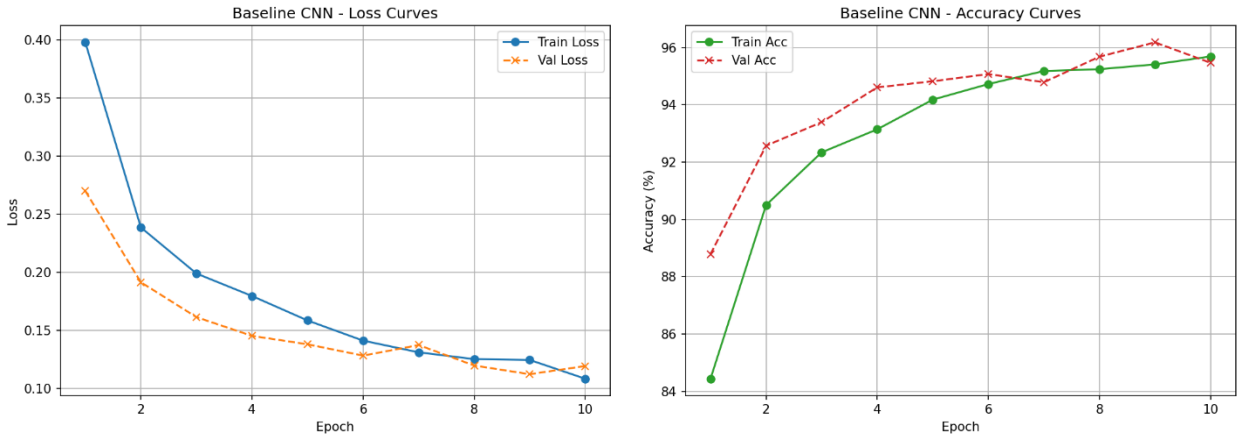

Fig. 12. Training and validation loss and accuracy curves for the best Baseline CNN model

The Baseline CNN shows steady, monotonic improvement across epochs. Training loss decreases consistently from 0.398 at epoch 1 to 0.108 at epoch 10, and training accuracy rises from 84.4% to 95.7%, indicating that the model is learning effectively from scratch without signs of early stagnation. Validation accuracy follows a similar upward trend, reaching a peak of 96.2% at epoch 9 before a slight dip to 95.5% at epoch 10. The gap between training and validation loss remains small throughout, suggesting the dropout regularisation (p=0.5) is functioning as intended. Final test set performance is 95.46% accuracy and F1-score of 0.9537.

The confusion matrix on the test set (total 2800 samples: 659 fake, 2141 real) is as follows:

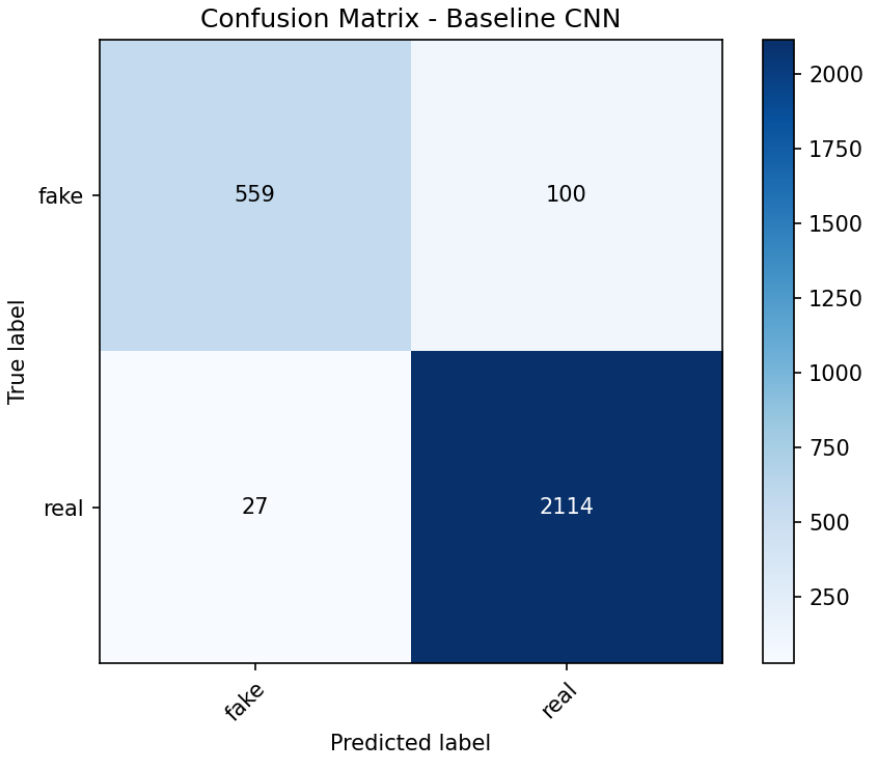

Fig. 13. Confusion matrix for the best Baseline CNN model

The model correctly classifies 559 out of 659 AI-generated images (true positives) and 2114 out of 2141 real images (true negatives). The primary error mode is false negatives (100 AI-generated images are incorrectly labelled as real). This is the more dangerous failure type in a detection context, as it allows synthetic content to pass undetected. False positives are considerably fewer at 27, meaning the model rarely flags a genuine image as AI-generated. The class imbalance in the test set (roughly 1:3 fake-to-real ratio) may contribute to this asymmetry, as the model has seen proportionally more real examples during training and is therefore slightly biased toward predicting real.

4.7.2 ResNet50

The table below reports training and validation loss and accuracy across all 10 epochs:

Epoch	Train Loss	Train Accuracy	Val Loss	Val Accuracy
1	0.1027	96.68%	0.0199	99.25%
2	0.0142	99.60%	0.0139	99.54%
3	0.0059	99.88%	0.0148	99.57%
4	0.0037	99.90%	0.0156	99.50%
5	0.0046	99.89%	0.0142	99.57%
6	0.0024	99.94%	0.0217	99.36%
7	0.0024	99.92%	0.0245	99.32%
8	0.0045	99.90%	0.0169	99.54%
9	0.0024	99.93%	0.0160	99.36%
10	0.0016	99.95%	0.0166	99.43%

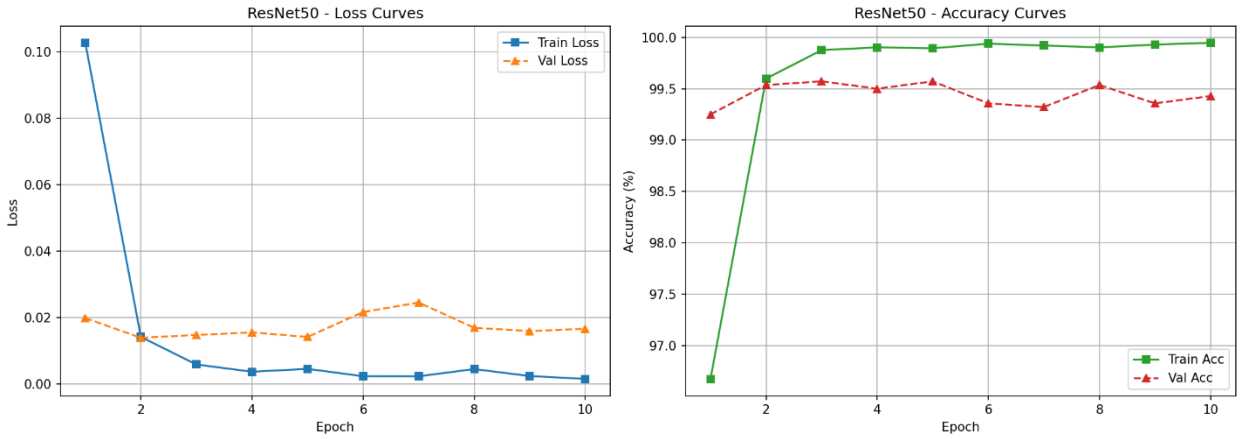

Fig. 14. Training and validation loss and accuracy curves for the best ResNet50 model

ResNet50 converges almost immediately. By epoch 1, validation accuracy already reaches 99.25% (a level the Baseline CNN never achieves even after 10 epochs). Training loss drops from 0.103 to 0.002 across the run, while validation loss stabilises around 0.014–0.025 from epoch 2 onwards with no clear upward trend, indicating the model is not overfitting despite near-perfect training accuracy. This convergence behaviour is a direct consequence of transfer learning: pretrained ImageNet weights already encode rich, generalisable visual features, so only the fine-tuned layer4 block and the new classification head require meaningful updates. Final test set performance is 99.43% accuracy and F1-score of 0.9943.

The confusion matrix on the test set (total 2800 samples: 659 fake, 2141 real) is as follows:

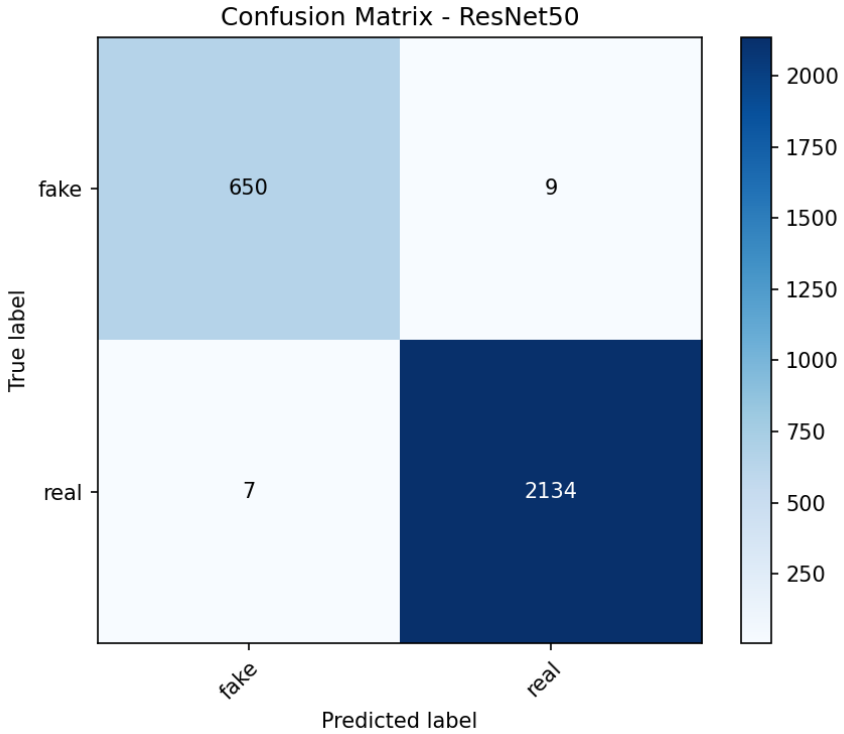

Fig. 15. Confusion matrix for the best ResNet50 model

ResNet50 correctly classifies 650 out of 659 AI-generated images and 2134 out of 2141 real images. Both error types are minimal: only 9 false negatives (AI-generated images classified as real) and 7 false positives (real images classified as AI-generated). Compared to the Baseline CNN, false negatives are reduced by 91% (from 100 to 9) which is the most important practical improvement, as it means the detector is far less likely to miss synthetic content. The near-symmetry of errors (9 vs 7) also suggests the model is not biased toward either class, which is consistent with the high F1-score of 0.9943.

5. Summary, Conclusions, and Future Work

5.1 Summary

This project developed and evaluated a deep learning pipeline for binary classification of AI-generated images in a social media context. Two architectures were compared (a custom Baseline CNN and a fine-tuned ResNet50) across a comprehensive set of parameter conditions and seven distinct datasets. The best-performing configuration achieves around 99% accuracy on in-distribution data, while external generalisation drops substantially - reaching at most around 70%, and often significantly lower for datasets produced by unseen or stylistically divergent generators.

5.2 Key Conclusions

- Transfer learning decisively outperforms training from scratch. ResNet50 with fine-tuning consistently exceeds the Baseline CNN on every metric and dataset combination, while requiring fewer epochs and being more robust to parameter variation.
- Cross-dataset generalisation is the primary challenge. Models trained on a single dataset exhibit strong in-distribution performance but degrade sharply when applied to images from different generators (Kaggle 1 accuracy dropped to 47.27% when training solely on PARVESHI).
- Multi-dataset training is essential for deployment robustness. Combining training sources substantially narrows the generalisation gap, lifting worst-case accuracy from ~47% to ~66% on the most challenging dataset.
- Higher input resolution improves detection quality. Increasing image size from 224 to 256/299 px enables the network to detect fine-grained artefacts characteristic of generative models, yielding up to +2.25 pp accuracy improvement for the Baseline CNN.
- Learning rate stability is a transfer learning advantage. ResNet50 maintains consistent performance (~99%) across all tested learning rates, whereas the Baseline CNN collapses at rates above 0.001.
- Epoch count has limited impact on ResNet50 but matters for the Baseline CNN. ResNet50 reaches near-peak performance within 5 epochs and shows no improvement beyond 10, confirming that pretrained weights eliminate the need for extended training. The Baseline CNN continues to gain accuracy up to 15 epochs but never closes the gap with ResNet50.

5.3 Possible Improvements and Future Work

Future work could focus on expanding the training dataset with images generated by newer diffusion models, such as FLUX and Stable Diffusion 3, to improve the model's ability to recognize emerging generation artifacts and signatures. The robustness and reliability of the system could also be enhanced through ensemble learning, where predictions from multiple models trained on different data subsets are combined to achieve better generalization and calibration. Furthermore, the project could be extended beyond static image classification by incorporating video frame analysis and additional contextual information, such as upload history, account age, or other platform-level metadata, which may significantly improve detection.

6. References

[1] Parveshiiii. AI-vs-Real Dataset. Hugging Face.

https://huggingface.co/datasets/Parveshiiii/AI-vs-Real

[2] LuckyCow. Ai_vs_real_web_images Dataset. Hugging Face.
https://huggingface.co/datasets/LuckyCow/Ai_vs_real_web_images

[3] julienlucas. midjourney-dalle-sd-nanobananapro-dataset. Hugging Face.
https://huggingface.co/datasets/julienlucas/midjourney-dalle-sd-nanobananapro-dataset

[4] tristanzhang32. AI Generated Images vs Real Images. Kaggle.
https://www.kaggle.com/datasets/tristanzhang32/ai-generated-images-vs-real-images

[5] Hemg. AI-Generated-vs-Real-Images-Datasets. Hugging Face.
https://huggingface.co/datasets/Hemg/AI-Generated-vs-Real-Images-Datasets

[6] rhythmghai. AI vs Real Images Dataset. Kaggle.
https://www.kaggle.com/datasets/rhythmghai/ai-vs-real-images-dataset

[7] Mukherjee, S. The Annotated ResNet-50. Towards Data Science.

https://towardsdatascience.com/the-annotated-resnet-50-a6c536034758/

[8] PyTorch Documentation. https://pytorch.org/docs/

[9] Streamlit Documentation. https://docs.streamlit.io/

7. Project Repository

All source code is available at: https://github.com/Blazejoida/ai-image-detector